Lab 02 实验报告：File Tools（V2）

项目	内容
实验	Lab 02 — File Tools（在 V1 之上构建 V2）
工程	`agent-cli` （ `com.agent:agent-cli:0.1.0` ）
对应学习目标	LO2（Agent Tool 开发能力）、LO5（AI 辅助开发能力）
新增包	`com.agent.tool` （ `src/main/java` 、 `src/test/java` 各 4/5 个文件）
基线版本	V1 提交 `25ac774` （Lab01 完成，工作区干净）
本次提交	`7e7179d lab02: file tools`
操作系统	Windows 11（x64），PowerShell
工具链	JDK 22.0.2（以 `--release 17` 编译）、Maven 3.9.16、Git 2.47.1、AI Coding Agent
报告日期	2026-09-14

1. 实验目的

本实验在 V1 之上让 Agent 第一次获得「读取与搜索项目文件」的能力，达成的能力如下。

- **LO2（Agent Tool 开发能力）**
  - 用 Java NIO（ `Path` / `Files` ）而不是老的 `java.io.File` 完成文件操作；
  - 用 `Files.walk` 递归遍历目录，并用 try-with-resources 管理 `Stream<Path>` 底层句柄；
  - 用 `Files.readString` / `Files.newBufferedReader` 读取文本，并用 `@TempDir` 构造隔离的测试夹具；
  - 处理文件异常：区分「参数非法」与「路径不存在或不是文件」，单个文件读取失败时跳过而不中断整体搜索；
  - 把功能封装为职责单一的独立 Tool，且三个 Tool 之间互不依赖。
- **LO5（AI 辅助开发能力）**
  - 用自然语言描述 Tool 的需求并让 AI 生成代码；
  - 审查 AI 生成的文件操作代码，识别出**忽略规则硬编码、平台分隔符、静默吞异常、重载歧义**等问题；
  - 用 `@TempDir` 单元测试与命令行 Demo 双重验证 Tool，并记录 `Prompt → AI 输出 → 人工审查 → 测试验证 → 修改` 的完整证据。

一句话总结：Lab 01 证明了「AI 写的代码可以被测试验证」，Lab 02 开始验证「AI 写的**文件操作**代码是否真的安全、可移植、可维护」。

2. 实验环境与起步检查

环境继承 Lab01，无新增依赖，全部使用 JDK 内置的 `java.nio.file` ：

```
java -version   ->  java version "22.0.2" 2024-07-16
mvn -version    ->  Apache Maven 3.9.16 (Java version: 22.0.2)
git --version   ->  git version 2.47.1.windows.1
```

起步检查（对应实验 §4.3）：

检查项	结果
`cd` 到 V1 项目目录	✅ `E:\agent-cli`
工作区是否干净	✅ `git status --short` 无输出，HEAD = `25ac774`
基线测试	✅ Lab01 的 `AgentTest` 9 个用例在本次第一次 `mvn clean test` 中与新增用例一并执行、全部通过（见 §6.1）
构建是否使用 release 17	✅ 日志 `Compiling 7 source files with javac [debug release 17]`

3. 实现过程

步骤	动作	产出 / 证据
Step 1	新建 `com.agent.tool` 包（主源码与测试各一个）	`src/main/java/com/agent/tool/` 、 `src/test/java/com/agent/tool/`
Step 2	实现 `ListFilesTool` （AI 生成 → 审查 → 修改）	递归列出 + 忽略规则 + 深度限制 + 树形输出
Step 3	实现 `ReadFileTool`	UTF-8 读取 + `NoSuchFileException` + 流式分页
Step 4	实现 `SearchCodeTool`	遍历子目录 + 跳过二进制 + 文件 + 行号 + 内容
Step 5	编写 `@TempDir` 测试	4 个 Tool 测试类共 32 个 `@TempDir` 用例（另有控制台编码与 CLI 用例，见 §6.2）
Step 6	构建与验证	`mvn clean test` → `Tests run: 52, Failures: 0, Errors: 0` ， `BUILD SUCCESS`
Step 7	运行 `FileToolsDemo` 做命令行验收	列出 17 个文件、读取 `README.md` 、搜到 24 行 `TODO`
Step 8	Git 增量提交	`7e7179d lab02: file tools`

补充实现（选做 / 挑战任务，详见 §10）：

- `IgnoreRules` ： 把忽略规则从硬编码常量搬到 `.agentignore` 配置文件（选做 3）；
- `listFiles(root, maxDepth)` ： 限制递归深度（选做 1）；
- `searchCodeRegex(Pattern, root)` ： 正则搜索（选做 2）；
- `tree(root)` ： 树形缩进输出（挑战 1）；
- `readFile(path, offset, limit)` ： 流式分页读取（挑战 2）。

最终结构（与实验 §7.1 一致，另加 `IgnoreRules` 与 `.agentignore` ）：

```
agent-cli/
├── .agentignore                    ← 忽略规则配置（新增）
├── .gitignore
├── pom.xml
├── README.md
├── Lab01-Report.md
├── Lab02-Report.md
└── src/
    ├── main/java/com/agent/
    │   ├── Agent.java
    │   ├── Main.java
    │   └── tool/
    │       ├── FileToolsDemo.java
    │       ├── IgnoreRules.java
    │       ├── ListFilesTool.java
    │       ├── ReadFileTool.java
    │       └── SearchCodeTool.java
    └── test/java/com/agent/
        ├── AgentTest.java
        └── tool/
            ├── IgnoreRulesTest.java
            ├── ListFilesToolTest.java
            ├── ReadFileToolTest.java
            └── SearchCodeToolTest.java
```

4. 核心代码说明

4.1 三个 Tool 的职责与返回类型

Tool	方法	返回类型	职责	异常契约
ListFilesTool	listFiles(Path)	List<String>	递归列出文件，相对路径 + / 分隔 + 字典序	root 不是目录 → NoSuchFileException
	listFiles(Path, int)	List<String>	限制递归层数（1 = 直接子项）	maxDepth < 0 → IllegalArgumentException
	tree(Path)	List<String>	树形缩进输出，目录以 / 结尾	同上
ReadFileTool	readFile(Path)	String	读取整个 UTF-8 文本文件	非普通文件 → NoSuchFileException；非法 UTF-8 → MalformedInputException
	readFile(Path, long, long)	String	流式分页读取（offset + limit）	offset < 0 或 limit <= 0 → IllegalArgumentException
SearchCodeTool	searchCode(String, Path)	List<Match>	子串匹配，返回文件 + 行号 + 内容	空关键字 → IllegalArgumentException；非目录 → NoSuchFileException
	searchCodeRegex(Pattern, Path)	List<Match>	正则匹配	pattern == null → NullPointerException

Match 用 record 定义：

```
/** 一条命中记录：文件（相对路径）、行号（1 起）、该行内容（已 trim）。 */
public record Match(String file, int line, String content) {
}
```

4.2 关键实现片段（含取舍原因）

ListFilesTool：关闭 Stream + 过滤目录 + 统一分隔符

```
try (Stream<Path> stream = Files.walk(root, maxDepth)) {
    return stream
            .filter(Files::isRegularFile)                      // 只要文件，不要目录
            .filter(path -> !ignoreRules.isIgnored(path, root))  // 跳过 target/、.git/、*.class
            .map(path -> toRelativeString(root, path))          // 相对路径 + 统一 '/'
            .sorted()                                           // 结果可断言、可复现
            .toList();
}
```

> Path::toString 在 Windows 上返回 src\main\A.java，在 macOS 上返回 src/main/A.java。
> 对「喂给 LLM 的文本」和「测试断言」来说，这种平台差异是纯粹的噪声，因此在 Tool 边界上统一成 /。

ReadFileTool：先判类型，再读内容；分页用 BufferedReader 流式跳过

```
public String readFile(Path path) throws IOException {
    requireRegularFile(path);          // 文件不存在或不是普通文件 → NoSuchFileException
    return Files.readString(path);     // UTF-8：非法字节抛 MalformedInputException
}

public String readFile(Path path, long offset, long limit) throws IOException {
    // ... 参数校验后：
    try (BufferedReader reader = Files.newBufferedReader(path)) {
        String line;
        long lineNumber = 0;
        while ((line = reader.readLine()) != null) {
            if (lineNumber >= offset) {
                if (slice.size() >= limit) break;
                slice.add(line);
            }
            lineNumber++;
        }
    }
    return String.join("\n", slice);
}
```

内存占用与 `limit` 相关，而**不与文件大小相关**：这正是分页相比 `Files.readString` 的价值。
返回内容统一用 `\n` 连接（而不是 `System.lineSeparator()`），同样是为了跨平台结果一致。

SearchCodeTool：遍历 → 跳过 → 逐行匹配 → 收集命中

```
try (Stream<Path> stream = Files.walk(root)) {
    List<Path> files = stream
            .filter(Files::isRegularFile)
            .filter(path -> !ignoreRules.isIgnored(path, root))
            .sorted()
            .toList();                          // 先关闭 Stream（释放目录句柄），再逐个读文件
    for (Path file : files) {
        searchInFile(lineMatcher, root, file, result);
    }
}


private void searchInFile(...) {
    try (BufferedReader reader = Files.newBufferedReader(file)) {
        String line;
        int lineNumber = 0;
        while ((line = reader.readLine()) != null) {
            lineNumber++;
            if (lineMatcher.test(line)) {
                result.add(new Match(relativize(root, file), lineNumber, line.trim()));
            }
        }
    } catch (IOException skipped) {
        // 二进制文件、无权限文件、非法 UTF-8：跳过该文件，不让整次搜索失败。
    }
}
```

4.3 为什么此刻还不统一接口

实验明确指出这是**刻意保留的技术债**：三个 Tool 的方法签名各不相同（`List<String>` / `String` / `List<Match>`，参数顺序与个数也不一致）。本实验保留它，是因为痛点只有在**真正尝试统一调用**时才会暴露：

1. 想写出 `tools.forEach(t -> t.execute(...))` 这样的统一循环，会发现三个方法连参数个数都不同；
2. 想把工具清单暴露给 LLM（工具名 + 描述 + 参数 schema），会发现没有统一的入口来生成描述；
3. 想统一错误处理与权限控制，会发现每个 Tool 的异常契约都不一样。

Lab04 将据此抽象出 `Tool` 接口（如 `String execute(String args)`），把上面三条痛点一次性解决。就本实验而言，**职责单一**比**形式统一**更重要：先把三种能力做对、做可测，再谈抽象。

5. AI 辅助开发过程记录

5.1 AI Task 01：ListFilesTool

Prompt

> 使用 Java NIO（Files.walk）实现一个递归列出目录文件的 Tool，返回相对路径列表，注意关闭 Stream。

AI 输出（关键片段）

```
public List<String> listFiles(Path root) throws IOException {
    try (Stream<Path> stream = Files.walk(root)) {
        return stream
            .filter(Files::isRegularFile)
            .map(root::relativize)
            .map(Path::toString)
            .sorted()
            .toList();
    }
}
```

人工审查（对照三问：是否递归？是否过滤目录？是否关闭 Stream？）

审查项	结论
是否递归	✅ `Files.walk` 默认递归到底
是否只保留文件	✅ `filter(Files::isRegularFile)`
是否关闭 Stream	✅ try-with-resources
相对路径化	✅ `root.relativize`
是否需要忽略 `target/`、`.git/`	❌ **缺失**：会把构建产物与版本库元数据一起列出来
返回值是否跨平台稳定	❌ `Path::toString` 在 Windows 上产生 `\`
输入非法时的行为	⚠️ 未校验 `root` 是否为目录

我的修改

1. 引入 `IgnoreRules`（选做 3）：`target/`、`.git/`、`.idea/`、`node_modules/` 等目录，以及 `*.class`、`*.jar`、`*.png` 等二进制/归档扩展名，全部搬到 `.agentignore` 配置文件，Tool 启动时加载（文件缺失则回退 `IgnoreRules.defaults()`）。
2. 相对路径统一 `/` 分隔符，使测试断言与 Demo 输出跨平台一致。
3. 增加 `requireDirectory(root)`，`root` 不存在或不是目录时抛出带明确中文说明的 `NoSuchFileException`。
4. 选做 1：增加 `listFiles(root, maxDepth)`；挑战 1：增加 `tree(root)`。

测试验证：`ListFilesToolTest` 8 个用例（递归/排序/只列文件/忽略规则/深度限制/负参数/非法目录/树形输出/自定义规则注入）全部通过。

5.2 AI Task 02：ReadFileTool

Prompt

> 实现 ReadFileTool，读取指定文本文件并返回内容，处理文件不存在的情况。

AI 输出

```
public String readFile(Path path) throws IOException {
    if (!Files.isRegularFile(path)) {
        throw new NoSuchFileException("不是文件或不存在: " + path);
    }
    return Files.readString(path);
}
```

人工审查：File Not Found / Encoding / Exception / Large File 四项

关注点	AI 的处理	我的判断与修改
File Not Found	`NoSuchFileException`	✅ 合理（理由见下）
路径是目录	`isRegularFile` 为 false 也会抛同一异常	✅ 接受，但补测试显式固化这一行为
Encoding	`Files.readString` 默认 UTF-8	✅ 接受；非法字节抛 `MalformedInputException` 而非静默替换，符合「宁可报错也不返回错数据」
Large File	无处理	⚠️ 改进：增加挑战 2 的 `readFile(path, offset, limit)`

异常类型的选择：`NoSuchFileException` 而不是 `IllegalArgumentException`

这是本次审查中最需要想清楚的一处：

1. **语义匹配**：`NoSuchFileException` 表达的是「这个路径不存在」，而 `IllegalArgumentException` 表达的是「你传进来的参数本身不合法」。路径 `not-exist.txt` 作为**参数**是完全合法的，只是**运行时状态**不满足要求。
2. **可捕获性**：`NoSuchFileException` 是 `IOException` 的子类，调用方既可以精确 `catch (NoSuchFileException e)` 给出「文件不存在，请检查路径」的提示，也可以统一 `catch (IOException e)` 兜底；用 `IllegalArgumentException`（RuntimeException）则会把「外部世界的问题」和「程序 bug」混在一起。
3. **与其它 Tool 的一致性**：`ListFilesTool` 与 `SearchCodeTool` 对「目录不存在」也抛 `NoSuchFileException`，三个 Tool 的异常语义因此保持一致。

同时保留 `IllegalArgumentException` 用于**真正的参数错误**，例如 `limit <= 0`、`maxDepth < 0`、`keyword` 为空 —— 这类错误不是运行环境造成的，而是调用方写错了。

测试验证：`ReadFileToolTest` 9 个用例，其中 4 个是异常路径（文件不存在、路径是目录、非法 UTF-8、参数非法）。

5.3 AI Task 03：SearchCodeTool

Prompt

> 实现 SearchCodeTool，在指定目录中搜索包含指定字符串的代码文件，返回文件、行号、内容；跳过二进制文件；单文件读取失败不应中断搜索。

AI 输出（关键片段）

```
private static final List<String> IGNORED_EXT =
    List.of(".class", ".png", ".jpg", ".jar", ".DS_Store");

private void searchInFile(String keyword, Path root, Path p, List<Match> result) {
    try {
        List<String> lines = Files.readAllLines(p);
        for (int i = 0; i < lines.size(); i++) {
            if (lines.get(i).contains(keyword)) {
                result.add(new Match(root.relativize(p).toString(), i + 1, lines.get(i).trim()));
            }
        }
    } catch (IOException ignored) {
        // 跳过无法按文本读取的文件
    }
}
```

人工审查（逐条对照实验要求）

要求	AI 是否满足	发现的问题
遍历子目录	✅ `Files.walk`	—
跳过二进制文件	⚠️ 部分满足	只看**扩展名**，目录级的 `target/` 、 `.git/` 没有跳过；本项目 `target/classes/**` 里的产物会被搜索并产生噪声命中
返回文件 + 行号 + 内容	✅	行号从 1 开始、内容 `trim()` ，正确
单文件失败不中断	✅	行为正确，但 `catch (IOException ignored)` 完全静默，出问题时无法排查
忽略规则的表达	❌	`IGNORED_EXT` 里混进了 `.DS_Store` ——它不是扩展名而是完整文件名；靠 `endsWith` 恰好能匹配，但规则语义已经混乱
结果顺序	⚠️	未排序，命中顺序取决于文件系统的遍历顺序，测试难断言
相对路径分隔符	❌	同样存在 Windows `\` 问题
正则支持（选做 2）	❌ 未提供	直接加重载 `searchCode(Pattern, ...)` 会让 `searchCode(null, root)` 产生歧义

我的修改

1. 忽略规则统一为 `IgnoreRules` 的两种语义：**目录名规则**（`target/` ，连同内容一起跳过）与 **glob 规则**（`*.class` ），并把 `.DS_Store` 这类完整文件名也按 glob 处理，规则语义不再混淆。
2. 遍历结果显式 `sorted()` ，保证命中顺序可复现。
3. 用 `BufferedReader` 逐行读取替代 `Files.readAllLines` ，避免一次性把整个文件读入内存。
4. 保留「跳过失败文件」的语义但写明注释，并用测试固化： `shouldSkipUndecodableFileAndKeepSearching` 写入含 `0xFF 0xFE` 非法字节、同时包含 `TODO` 的文件，断言搜索仍然只命中其它文件而不抛异常。
5. 选做 2 的正则支持命名为 `searchCodeRegex(Pattern, Path)` 而不是重载 —— 因为 `searchCode(String, Path)` 与 `searchCode(Pattern, Path)` 并存时， `searchCode(null, root)` 在编译期就是歧义调用。**这本身就是「签名不统一」的一个具体症状**，正好为 Lab04 提供论据。
6. 相对路径统一 `/` 。

测试验证： `SearchCodeToolTest` 10 个用例（命中细节、子目录、同行多处、二进制扩展名、忽略目录、不可解码文件、trim、正则、空关键字、非法目录）全部通过。

6. 测试结果

6.1 `mvn clean test` 输出

```
[INFO] Compiling 8 source files with javac [debug release 17] to target\classes
[INFO] Compiling 7 source files with javac [debug release 17] to target\test-classes
[INFO] Running com.agent.AgentTest
[INFO] Tests run: 9, Failures: 0, Errors: 0, Skipped: 0 -- in com.agent.AgentTest
[INFO] Running com.agent.ConsoleStreamsTest
[INFO] Tests run: 6, Failures: 0, Errors: 0, Skipped: 0 -- in com.agent.ConsoleStreamsTest
[INFO] Running com.agent.MainTest
[INFO] Tests run: 6, Failures: 0, Errors: 0, Skipped: 0 -- in com.agent.MainTest
[INFO] Running com.agent.tool.IgnoreRulesTest
[INFO] Tests run: 5, Failures: 0, Errors: 0, Skipped: 0 -- in com.agent.tool.IgnoreRulesTest
[INFO] Running com.agent.tool.ListFilesToolTest
[INFO] Tests run: 8, Failures: 0, Errors: 0, Skipped: 0 -- in com.agent.tool.ListFilesToolTest
[INFO] Running com.agent.tool.ReadFileToolTest
[INFO] Tests run: 9, Failures: 0, Errors: 0, Skipped: 0 -- in com.agent.tool.ReadFileToolTest
[INFO] Running com.agent.tool.SearchCodeToolTest
[INFO] Tests run: 10, Failures: 0, Errors: 0, Skipped: 0 -- in com.agent.tool.SearchCodeToolTest
[INFO] Results:
[INFO] Tests run: 52, Failures: 0, Errors: 0, Skipped: 0
[INFO] BUILD SUCCESS
```

6.2 用例分布与覆盖说明

测试类	用例数	覆盖内容
`AgentTest` （Lab01）	9	回归： `Agent.respond` 的分支与边界未被改动破坏
`ConsoleStreamsTest` （编码）	5	编码选择规则：显式指定优先、空白/非法名字忽略、默认采用 JVM 探测值（GBK 与 UTF-8 两种情形）、探测值不可用时回退、 `-Dagent.encoding` 生效
`MainTest` （Lab01 修复）	6	用字节流模拟 GBK / UTF-8 控制台：中文输入正确回显、逐行处理、 `quit` （含大小写）、EOF 无输出
`IgnoreRulesTest`	5	目录规则、glob 规则、注释/空行跳过、配置文件缺失回退默认、从文件加载
`ListFilesToolTest`	8	递归、排序、只列文件、默认忽略规则、深度限制、负深度、非法根目录、树形输出、自定义规则注入
`ReadFileToolTest`	9	整文件读取、UTF-8、文件不存在、路径是目录、非法 UTF-8、分页切片、limit 超尾部、offset 越界、参数非法
`SearchCodeToolTest`	10	命中细节（文件/行号/内容）、子目录、同一文件多处、二进制扩展名、忽略目录、不可解码文件、trim、正则、空关键字、非法目录

6.3 `@TempDir` 夹具与异常用例说明

所有 Tool 测试都用 `@TempDir` 提供的**临时目录**作为夹具，而不是依赖工程里真实存在的文件：

```
@Test
void shouldFindKeywordWithLineNumberAndContent(@TempDir Path dir) throws IOException {
    Files.writeString(dir.resolve("A.java"), "public class A {\n    // TODO fix me\n}\n");

    List<SearchCodeTool.Match> matches = tool.searchCode("TODO", dir);

    assertEquals(List.of(new SearchCodeTool.Match("A.java", 2, "// TODO fix me")), matches);
}
```

三个「异常路径」用例最值得记录，因为它们验证的正是实验要求的鲁棒性：

用例	构造的输入	期望行为	为什么重要
`shouldThrowNoSuchFileExceptionWhenMissing`	指向不存在文件的 `Path`	抛 `NoSuchFileException` ，消息含路径	AT04 的自动化版本，避免「静默返回空串」这类危险实现
`shouldThrowOnNonUtf8Content`	写入 `0x89 P N G 0xFF 0xFE 0x00` 的字节	抛 `MalformedInputException`	若用「替换字符」策略，二进制会被当作文本返回，Agent 会拿到垃圾数据还以为读取成功
`shouldSkipUndecodableFileAndKeepSearching`	同时存在正常文件与非法字节文件，二者都含 `TODO`	只命中正常文件，不抛异常	验证「单文件失败 → 跳过 → 搜索继续」而非整体中断

> 这四个 Tool 测试类（合计 32 个用例）连同编码 / CLI 用例，在 0.2 秒左右即全部跑完（单类耗时约 0.008s–0.078s），且不写任何工程文件、不产生 `git status` 噪声 —— 这正是 Lab01 思考题里「可测试性是设计出来的」在文件操作场景下的延续。

7. 验证记录（对照实验 §9）

7.1 功能验证

编号	验收项	实际命令 / 方式	实际结果
AT01	列出文件	`FileToolsDemo` （ `listFiles(Path.of("."))` ）	输出 17 个文件（当时的工作区快照），相对路径、无 `targ`

编号	验收项	实际命令 / 方式	实际结果
AT02	读取文件	`FileToolsDemo README.md`	完整打印 `README.md` 内容（含中文）
AT03	搜索代码	`FileToolsDemo README.md TODO` （ `searchCode("TODO", Path.of("."))` ）	命中 24 行，格式 `文件:行号: 内容`
AT04	异常处理	`FileToolsDemo not-exist.txt`	抛出 `java.nio.file.NoSuchFileException: 不是文件或不存在: no` 进程退出码 1，无其它崩溃
AT05	Tool 独立性	检查三个 Tool 的相互引用	`ListFilesTool` / `ReadFileTool` / `SearchCodeTool` 之间**零引** `FileToolsDemo` 组装它们， `IgnoreRules` 是共享配置而非

AT03 的真实输出节选（ `文件:行号: 内容` ）：

```
src/main/java/com/agent/tool/FileToolsDemo.java:46: String keyword = rest.size() >= 2 ? rest.get(1) : "TODO";
src/test/java/com/agent/tool/SearchCodeToolTest.java:27: assertEquals(List.of(new SearchCodeTool.Match("A.java", 2, "// TODO fix me")), matches);
...
== 命中 24 行 ==
```

7.2 单元测试验证

见 §6.1： `Tests run: 52, Failures: 0, Errors: 0` + `BUILD SUCCESS` ，可重复执行。

7.3 命令行验证（清理并重编 + Demo）

```
Remove-Item target -Recurse -Force
mvn -q -DskipTests compile
java -cp target/classes com.agent.tool.FileToolsDemo README.md TODO
```

`mvn clean test` 已包含「删除 `target/` → 全量重编」的等价流程；Demo 的三段输出与 §7.1 一致。
另外验证了两个扩展开关：

```
java -cp target/classes com.agent.tool.FileToolsDemo --tree
== stdout encoding: UTF-8 ==
== list_files (--tree) ==
.agentignore
.gitignore
Lab01-Report.md
pom.xml
README.md
src/
  main/
    java/
      com/
        agent/
          Agent.java
          Main.java
          tool/
            ...
```

7.4 AI Agent 验证（实验 §9.4）

> **AI 的回答**
>
> 三个 Tool 的方法签名差异体现在五处：
>
> 1. **返回类型不同**： `ListFilesTool.listFiles` 返回 `List<String>` ， `ReadFileTool.readFile` 返回 `String` ， `SearchCodeTool.searchCode` 返回 `List<Match>` ；
> 2. **参数个数不同**： `listFiles(Path)` / `readFile(Path)` 只接受路径，而 `searchCode(String keyword, Path root)` 需要业务参数；
> 3. **参数顺序不同**： `searchCode` 把 `keyword` 放在 `Path` 之前，与另外两个「路径是唯一参数」的形态不一致；

> 4. **方法名不统一**：若要支持正则，只能叫 `searchCodeRegex`，因为 `searchCode(String)` 与 `searchCode(Pattern)` 重载会让 `searchCode(null, root)` 编译期歧义；
> 5. **异常契约不同**：`readFile` 对文件抛 `NoSuchFileException`，`listFiles` / `searchCode` 对目录抛 `NoSuchFileException`，而 `limit`、`maxDepth`、`keyword` 这类参数错误抛 `IllegalArgumentException`。
>
> 当 Tool 数量增多后，这种不一致会成为问题：Agent 无法用统一循环或注册表来调用工具（`Map<String, Tool>` 需要一个共同的类型）；无法统一生成给 LLM 看的工具描述与参数 schema；无法统一做错误处理、权限校验与审计；每新增一个工具都要回到 `Agent` 里写一个特例分支，违反开闭原则。这正是 Lab04 要抽象 `Tool` 接口的动机。

学生核对（逐条对照真实代码）

AI 的说法	代码证据	是否一致
返回类型三种	`List<String>` / `String` / `List<Match>`（`SearchCodeTool.Match` 为 record）	✅
参数个数不同	`listFiles(Path)`、`readFile(Path)`、`searchCode(String, Path)`	✅
参数顺序不同	`searchCode(String keyword, Path root)` 中关键字在前	✅
方法名 `searchCodeRegex` 用于规避重载歧义	代码注释与 §5.3 修改记录第 5 条	✅
异常契约不同	三类异常分别对应参数错误与路径错误，见 §4.1 表格	✅

核对结论：AI 的解释与代码完全一致，且准确点出了「签名不统一」这一为 Lab04 准备的痛点。

8. 遇到的问题与解决方案

问题 1：硬编码忽略规则导致搜索被构建产物淹没

现象：最初的实现只按扩展名忽略二进制文件，没有跳过目录。对真实的 Maven 项目运行搜索时，`target/classes/**`、`target/test-classes/**` 下的产物都会参与搜索，命中里混入大量与源码无关的结果。

根因：把「哪些路径不该看」当作代码里的常量而不是项目配置；而且「目录」与「文件扩展名」被塞进了同一个 `List<String>`，语义混乱（`.DS_Store` 其实是完整文件名）。

解决：抽出 `IgnoreRules`，用 `.agentignore` 配置两种规则语义：

```
target/          ← 以 / 结尾 = 目录名规则：该目录及其内容全部跳过
*.class          ← glob 规则：匹配文件名与相对路径
```

验证：`ListFilesToolTest.shouldApplyDefaultIgnoreRules` 与 `SearchCodeToolTest.shouldSkipIgnoredDirectories` 断言 `target/classes/Generated.java`（虽然扩展名是 `.java`，但位于被忽略目录内）不会出现在结果里。

问题 2：`Path::toString` 让输出带平台分隔符

现象：同一份测试在 Windows 上得到 `src\main\A.java`，在 macOS 上得到 `src/main/A.java`；断言必须写平台相关的字符串。

根因：`Path` 是平台抽象，但 `Path.toString()` 会使用当前平台的默认分隔符。

解决：在 Tool 的**输出边界**统一规范化：`relative.toString().replace('\\', '/')`。`Path` 在内部仍是平台无关的抽象（这是 NIO 的价值），只在面向外部（测试断言、LLM 文本、日志）时才统一成 `/`。

问题 3：把「目录不存在」当成「参数非法」是不合适的

现象：尝试用 `IllegalArgumentException` 表达「文件不存在」时，异常语义与调用方的处理方式都对不上：这是运行时环境问题（RuntimeException 表示程序 bug），不是调用方传错了参数。

解决：明确两类异常的边界——**路径/环境问题**用 `NoSuchFileException`（`IOException` 子类，可被 `catch (IOException)` 兜底），**真正的参数错误**（`limit <= 0`、`maxDepth < 0`、空关键字）用 `IllegalArgumentException`。并用测试把两类行为都固化。

问题 4： `catch (IOException ignored)` 静默吞异常

现象：AI 生成的 `searchInFile` 用 `catch (IOException ignored)` 跳过不可读文件 —— 行为符合「不中断搜索」的要求，但一旦出现意料之外的 I/O 问题（例如权限、编码），既没有日志也没有计数，排查时完全没有线索。

解决：本实验保留「跳过」语义（这是正确的设计：一次搜索跨越大量文件，不能因单个文件失败而整体失败），但把语义写进注释与报告，并用测试显式固化；如果后续要产品化，应改为「收集跳过原因并在结果中返回统计」。

问题 5：Windows GBK 控制台把中文输出显示成乱码

现象：Demo 打印 `README.md` 内容时，终端上是 `Lab 01 ▯▯ V1 ▯▯▯␣▯▯▯...`（用户实际操作时报告的现象）。

定位过程（用字节而不是肉眼判断）：把 Demo 的 stdout 重定向到文件后直接看原始字节，分别按两种编码解码同一行：

```
按 UTF-8 解码：Lab 01 ▯▯ V1 ▯▯▯␣▯▯▯▯▯▯h▯▯ Java 17 + Maven ...     ← 乱码
按 GBK   解码：Lab 01 的 V1 工程基础：一个 Java 17 + Maven ...     ← 正确
```

同时查看 JVM 属性：

```
file.encoding   = UTF-8     ← 文件读写用的字符集（JDK 18+ 固定为 UTF-8）
native.encoding = GBK
stdout.encoding = GBK       ← 关键：System.out 跟随控制台代码页
```

根因：文件本身是 UTF-8， `ReadFileTool` 也按 UTF-8 正确读取（ `shouldReadUtf8Content` 用例可证）；乱码只发生在**输出侧**——JDK 19 起 `System.out` 使用 `stdout.encoding`（Windows 上默认等于控制台代码页 GBK）而非 `file.encoding` ，于是 Java 写出 GBK 字节，而现代终端（Windows Terminal / VS Code / IDEA）按 UTF-8 解码。

第一次尝试（把输出固定为 UTF-8）：直接写入 `FileDescriptor.out` 以绕开 `System.out` 自带的编码器：

```
System.setOut(new PrintStream(
        new FileOutputStream(FileDescriptor.out), true, StandardCharsets.UTF_8));
```

字节比对确认输出已变成 UTF-8（按 UTF-8 解码正确、按 GBK 解码反而乱码）。**但这个修复并不完整**：在按 GBK 解码的真实 Windows 控制台里，中文又反过来变成乱码 —— 换句话说，**写死任何一种编码，都必然在另一种终端上出错**。

第二次尝试（猜测终端类型）：引入 `Console.isTerminal()` （JDK 22 新增，用反射调用以兼容 `--release 17` 编译）区分「真实终端」与「管道 / IDE 捕获」，并规定后者使用 UTF-8。

结果：仍然失败。 使用者反馈「默认仍然乱码，但加上 `-Dagent.encoding=GBK` 就正常」——
这说明那条启发式假设**不成立**： `输出被重定向 / 由 IDE 捕获` 并不等价于 `对端按 UTF-8 解码` 。
在中文 Windows 上，IDE 运行窗口捕获输出时对端往往仍按系统编码（GBK）解码。用「环境形态」去推测「对端解码方式」，本质上是在猜，而猜错时用户看到的就是乱码。

最终解决（不猜测，遵循 JVM 探测结果）：默认直接采用 JVM 探测到的编码，只在显式指定时覆盖：

```
static Charset charset(String forced, String detected, Charset fallback) {
    Charset chosen = resolve(forced);      // -Dagent.encoding=... （优先级最高）
    if (chosen == null) chosen = resolve(detected);   // stdin.encoding / stdout.encoding
    return chosen == null ? fallback : chosen;        // 兜底：native.encoding，再兜底 UTF-8
}
```

这样默认行为与「直接用 `System.out` 」完全一致（JDK 19+ 本来就是按控制台代码页 / 平台编码决定），
既不会在 GBK 控制台或 GBK 解码的 IDE 里出错，也不会在 UTF-8 终端、macOS / Linux 上出错；
确实需要不同编码时（例如 IDE 把运行窗口设成 UTF-8），再用系统属性显式声明。

验证（不加任何参数即为默认行为）：

```
本机属性：file.encoding=UTF-8  native.encoding=GBK  stdout.encoding=GBK
默认（无参数）：    == stdout encoding: GBK ==        → 按 GBK 解码，中文正确
显式覆盖：          -Dagent.encoding=UTF-8            → 输出 UTF-8 字节，按 UTF-8 解码正确
```

Demo 的第一行会打印实际生效的编码，出现乱码时可以据此立刻判断该用哪种覆盖值；选择规则本身由 `ConsoleStreamsTest` 的 5 个用例固定住。

> 补充：这套自适应逻辑后来被抽成公共类 `com.agent.ConsoleStreams`， `FileToolsDemo` 与本工程的 CLI（ `Main` ）共用 —— 因为 Lab01 的 CLI 存在**同一根因的输入侧缺陷**（ `new InputStreamReader(System.in)` 用默认 UTF-8 解码 GBK 控制台输入，中文变成 `???` ），详见 Lab01 报告「遇到的问题与解决方案」。

顺带记录过程中踩到的两个坑：① `-Dstdout.encoding=UTF-8` 在 PowerShell 下不加引号会被拆成两个参数（报 `ClassNotFoundException: /encoding=UTF-8` ），必须写成 `'-Dstdout.encoding=UTF-8'` ；② 用 `>` 重定向还会再经过 PowerShell 自身的一次编码转换，验证编码问题时应改用 `Start-Process -RedirectStandardOutput` 或直接比对字节。

结论：乱码是**输出编码**问题，不是 Tool 的数据问题。 `ReadFileTool` 的正确性由 `shouldReadUtf8Content` （ `"你好，Agent！"` ）用例保证；输出/输入编码则默认遵循 JVM 探测结果，并提供 `-Dagent.encoding` 覆盖。这一节也是本实验最有价值的一课：**面对「环境相关的编码」问题，宁可遵循平台探测结果，也不要写启发式猜测**。

问题 6： `Files.walk` 的 Stream 与文件句柄

现象/风险： `Files.walk` 返回懒加载的 `Stream<Path>` ，它持有目录句柄。若在 Stream 未关闭时就开始逐个读取（尤其是在 Windows 上）并做写入操作，容易出现句柄未释放导致的失败。

解决： `SearchCodeTool` 先用 `.toList()` 把路径全部收集完（此时 try-with-resources 保证 Stream 已关闭），再逐个读取文件； `ListFilesTool` / `tree` 也都在 try-with-resources 内完成终端操作。

问题 7：重载导致的调用歧义

现象：为支持正则，最自然的写法是增加重载 `searchCode(Pattern, Path)` 。但这样 `searchCode(null, root)` 在编译期就是歧义调用，而且对读代码的人来说「同一个方法名承担两种匹配语义」也不清晰。

解决：命名为 `searchCodeRegex(Pattern, Path)` 。这条不是「绕过去就算了」的补丁，而是记录了「签名不统一」痛点的又一个实例（见 §7.4）。

9. 思考题回答（实验 §10）

9.1 为什么「LLM 很聪明」并不意味着它能直接读取本地项目？这反映了 LLM 与外部世界怎样的边界？

因为「聪明」指的是它在**已进入上下文的信息**上做推理的能力，而不是**获取信息**的能力。LLM 的运行环境是一个推理进程：它只有一组 token（也就是上下文窗口），没有文件描述符、没有进程、没有网络 socket。你的磁盘路径 `E:\agent-cli\src\main\java\com\agent\Agent.java` 对模型来说只是一个字符串，除非有人把文件内容**读出来并放进上下文**。

这条边界可以概括为三点：

1. **边界是「上下文」而非「权限」**：模型不是「没权限读你的文件」，而是**没有任何读文件的通道**。即使给它完整的磁盘访问权，它也需要一个能执行 I/O 的程序，而它本身只输出文本。
2. **边界两侧的职责不同**：LLM 负责「决定读什么、怎么理解、下一步做什么」；Java 程序（Tool）负责「真正去文件系统执行并把结果变成文本」。
3. **边界带来了确定性的责任划分**：Tool 是确定性的、可单元测试的（本工程 52 个测试）；LLM 是概率性的、不可穷举的。把不确定性关在 `Agent` 的决策层，把确定性沉淀在 Tool 层，是整个 Agent 架构最重要的分界。

所以本实验实现的三个 Tool，本质上是 Agent 的「手脚」：**没有 Tool，再强的模型也只能凭空编造项目内容**（这也是幻觉的一大来源——模型被问到没见过的东西时，只能用训练分布里的典型代码去补全）。反过来，一旦有了 `ReadFileTool` 与 `SearchCodeTool` ，Agent 的输入就从「猜测」变成了「证据」。

9.2 三个 Tool 签名各不相同，当 Tool 越来越多时会带来什么问题？

具体会带来六类问题，而且都在 Tool 数量从 3 增加到 30 时集中爆发：

1. **无法统一注册与调用**： `Agent` 想维护一个工具清单（Lab04 的 Tool Registry 通常是 `Map<String, Tool>` ），但 `ListFilesTool` 、 `ReadFileTool` 、 `SearchCodeTool` 没有共同父类型，无法放进同一个集合，只能为每个工具写一个字段和一个分支： `if (name.equals("read_file")) ... else if (name.equals("search_code")) ...` 。新增工具必须修改 `Agent` ，违反开闭原则。
2. **无法给 LLM 统一暴露工具描述**：LLM 只能看到文本。要把「有哪些工具、每个工具要什么参数」告诉模型，需要为每个工具生成名称、描述与参数 schema；签名各异意味着没有统一的生成入口，只能手工维护一份「工具说明书」，而说明书和代码一旦不一致就是灾难。
3. **无法统一参数传递与结果序列化**：LLM 通常输出的是**文本**（如一段 JSON 或 `action: read_file\nargs: {...}` ）。Tool 侧若没有统一的 `execute(String args)` ，就需要为每个工具单独写「文本 → 参数对象」的解析代码；返回侧同样， `List<String>` / `String` / `List<Match>` 三种结构要

三种不同的渲染逻辑。

4. **无法统一错误处理与可观测性**：现在每个 Tool 的异常契约不同（谁抛 `NoSuchFileException`、谁抛 `IllegalArgumentException`），调用方无法写一段统一的 `try/catch`。工具一多，日志格式、耗时统计、失败计数都会各写一套。
5. **无法统一横切关注点**：安全是典型例子。Tool Framework 需要统一做「路径越权校验（禁止逃出项目根目录）」「危险操作二次确认」「调用审计」。这些逻辑只有在所有工具共享一个调用入口时才能集中实现，否则要在每个 Tool 里复制一遍。
6. **测试与文档成本线性增长**：每个工具都有各自的调用方式，集成测试要一个一个手写；文档要一页一页维护。

本实验刻意保留这种不一致，就是为了让 Lab04 的抽象**由痛点驱动**：先感受到「`tools.forEach(...)` 写不出来」，再认可 `Tool` 接口不是形式主义，而是把「调用契约」从 N 份收敛成 1 份。

9.3 统一接口应该长什么样？ `execute` 的参数与返回值如何同时容纳列表、字符串、匹配结果？

一个能容纳现有三个 Tool 的统一接口，大致是这样的：

```
public interface Tool {
    String name();                      // "read_file"，供 LLM 与注册表使用
    String description();               // 给 LLM 看的自然语言描述

    /** 入参统一为文本（JSON 或 "key=value"），出参统一为文本。 */
    ToolResult execute(String args);
}

public record ToolResult(boolean success, String output, String error) {
}
```

设计要点与理由：

1. **参数统一为 `String`**：因为调用链的另一端是 LLM，模型只能产出文本。把文本解析成参数对象的责任放在工具内部（例如 `read_file` 解析 JSON `{"path":"README.md","offset":0,"limit":40}`，`search_code` 解析 `{"keyword":"TODO","root":"."}`），接口就与「LLM 是文本输入」这一事实对齐。代价是工具内部要写解析与校验，收益是调用方（Agent）无需知道任何具体参数结构。
2. **返回值统一为文本**：三种返回结构都需要**序列化**：
   - `List<String>`（文件列表）→ 每行一个路径，例如 `src/main/java/com/agent/Agent.java`；
   - `String`（文件内容）→ 原样返回，建议附加行号前缀（如 `12: ...`）或头尾截断提示，便于模型定位；
   - `List<Match>`（搜索结果）→ 逐行渲染成 `文件:行号: 内容`，这正是 `FileToolsDemo` 已经在使用的格式。

   也就是说，**统一接口的返回值不是「把三种类型塞进一个泛型」，而是「定义一个稳定的文本协议」**。这是刻意的取舍：牺牲类型安全，换取与 LLM 的无缝对接与统一的渲染逻辑。
3. **同时返回成功/失败而不是抛异常**（或用 `ToolResult` 包一层）：Agent 循环不应该因为一个工具失败而崩溃，它需要把失败信息作为「观察结果」喂回给 LLM，让模型决定换工具或换参数。把 `IOException` 转成 `ToolResult(success=false, error=...)` 是把「异常」升级为「对话的一部分」。
4. **元信息（`name` / `description`）放进接口**：这样 Tool Registry 可以自动生成给 LLM 的工具清单，避免「代码与说明书不一致」。

这也是 Lab04 会遇到的真实张力：**类型安全 vs 统一调度**。本实验的三个 Tool 用「各自最自然的类型」表达返回值（可读、可测），Lab04 用「统一的文本协议」表达（可调度、可喂给 LLM）。两者都合理，区别在于服务的目标不同。

9.4 `record` 相比传统 POJO 带来了什么工程价值？

`SearchCodeTool.Match` 只有一行：

```
public record Match(String file, int line, String content) {
}
```

如果手写等价 POJO，需要：3 个 `private final` 字段、1 个三参构造器、3 个 getter、`equals`、`hashCode`、`toString` —— 大约 40 行，而这些代码**没有任何业务含义**。`record` 的价值有四层：

1. **语义即文档**：`record Match(String file, int line, String content)` 一眼说明「一条命中记录由这三样东西构成」。它声明的是**数据形状**，而不是一个可变的、有身份的对象。对「查询结果」这种值对象，这正是准确的建模。
2. **正确性是白送的**：`equals` / `hashCode` 自动按**分量**生成，因此本实验可以写出这样的断言：

   ```
   assertEquals(List.of(new SearchCodeTool.Match("A.java", 2, "// TODO fix me")), matches);
   ```

   如果 `equals` 忘了写或用 `Object` 的默认实现（比较引用），断言必然失败，而且失败原因很难察。`record` 让「值相等」成为默认行为，消除了这一类易错代码。

3. **不可变性**：所有分量都是 `final` 的，没有 setter。搜索结果在多个调用者之间共享时不会被意外篡改，天然线程安全。Java 的「不可变对象更安全」这一长期经验，被语言层面固化为默认选项。
4. **与模式匹配（Java 16+）配合**：`if (result instanceof Match(String file, int line, String content))` 可以直接解构，无需 getter —— 这在后续 ReAct 循环里处理「不同类型的观察结果」时会显著简化代码。

工程上更深的收益是**减少决策面**：传统 POJO 给了开发者无数选择（可变还是不可变？ `equals` 用不用 Lombok？要不要无参构造给框架用？），每个选择都可能出错。 `record` 把这些约定收回语言本身，代价是**不能继承、不能在 record 里放可变状态**——对于值对象这类场景，这个约束恰好是我们想要的。

9.5 「模型决定调用什么 Tool」与「Java 程序真正执行 Tool」是两个不同过程——本实验实现的是哪一半?

本实验实现的是**后半段：执行**。完整链路是：

```
① 用户输入需求
        ↓
② Agent 把「工具清单 + 描述」与需求一起发给 LLM      ← Lab03/Lab04
        ↓
③ LLM 输出「我要调用 read_file，参数是 {"path":"README.md"}」   ← 由模型决定（概率性）
        ↓
④ Agent 解析这段文本，找到对应工具
        ↓
⑤ 真正执行 Tool：读磁盘、返回文本                          ← ★ 本实验实现的就是这一步
        ↓
⑥ 把结果作为「观察」拼回上下文，回到 ②（ReAct 循环）
```

具体地，本实验交付的是第 ⑤ 步以及它依赖的全部确定性逻辑：

- `ListFilesTool.listFiles` / `tree`：真正遍历文件系统；
- `ReadFileTool.readFile`：真正读取文件内容；
- `SearchCodeTool.searchCode` / `searchCodeRegex`：真正遍历并逐行匹配；
- 跳过规则、编码处理、异常契约：让执行结果**可预测、可测试**。

而**尚未实现**的是第 ②③④⑥ 步：本实验里「调用哪个 Tool」仍然由人（`FileToolsDemo.main` 里硬编码 `args[0]` / `args[1]`）决定，而不是由模型决定；也没有把工具清单与描述暴露给 LLM。换句话说：

> 本实验造好了「手脚」并单独验证了它们，但「大脑决定用手还是用脚」的环节要到引入 LLM 之后才补齐。

这个划分有重要的工程意义：**执行层必须是确定且可测试的，决策层才允许是概率的**。因为执行层一旦出错（读错文件、路径越权、把二进制当文本），后果是真实且不可撤销的；而把执行层用 52 个测试钉死之后，后续引入 LLM 时，出问题就可以快速定位成「模型决策错了」而不是「工具执行错了」。

9.6 某个文件读取抛 `IOException` 时，为什么选择「跳过」而非「中断」？这对鲁棒性意味着什么?

因为两者的**失败半径**完全不同：

策略	一次搜索的结果	失败半径
中断（fail-fast）	抛异常，一行命中都拿不到	1 个坏文件 → 整次搜索失败
跳过（fail-soft）	返回其余文件的命中，坏文件被忽略	1 个坏文件 → 只损失该文件

「跳过」在本场景是正确的，理由有四：

1. **坏文件是常态而非异常**：真实项目里总有二进制文件（图片、字体、 `.jar` ）、锁文件、被占用或没有读权限的文件、非 UTF-8 的遗留文本（GBK 编码的旧文件）。如果每一次都要中断，搜索工具在真实项目上几乎不可用。
2. **搜索是「尽力而为」的语义**：调用方要的是「有哪些地方出现了这个关键字」，而不是「保证扫描了每一个字节」。少一个不可读文件的命中，不影响结论的可用性。
3. **Agent 的鲁棒性要求**：Agent 是在循环中自主行动的，一次工具调用失败会打断整个 ReAct 循环，浪费掉已经消耗的 token 与推理步骤。让工具「降级返回」比「抛出异常」更符合 Agent 的运行方式（后续 Lab04 更会把失败包装成 `ToolResult(success=false)` ，让模型自己决定下一步）。
4. **与「参数错误」区分开**：`keyword` 为空这类错误是调用方写错了，必须立刻报错（fail-fast）；而「某个文件读不了」是外部世界的不确定性，应当降级处理（fail-soft）。**同一系统里同时存在两种策略，关键在于区分「谁错了」。**

但「跳过」也有代价，必须正视：**搜索会静默地不完整**。用户不知道有 3 个文件被跳过了，可能因此得出错误结论。理想的实现应当「跳过 + 报告」：返回结果的同时附上「已跳过 N 个无法读取的文件（附路径列表）」，让调用方（或 LLM）自行判断结果可信度。本实验为了保持返回类型简单（ `List<Match>` ）暂未实现这一点，但已在 §8 问题 4 中记录为已知取舍。

对 Agent 鲁棒性的整体启示是：**把「局部失败」和「整体失败」分开设计**。局部失败（单个文件、单个工具调用）应当降级并留下痕迹；整体失败（参数非法、根目录不存在）应当立即报错。这样 Agent 才不会因为角落里的一颗石子而停在半路，也不会在数据不完整时毫不知情。

9.7 `Path` 抽象体现了哪条平台无关性原则？老版 `java.io.File` 在跨平台上有何不足？

`Path` 体现的是**「抽象泄漏最小化」/「把平台差异收敛到边界」的原则：程序内部用**平台无关的模型**（路径 = 由若干「名字」组成的序列 + 根）表达意图，只在真正与操作系统交互的边界上由 JVM 负责转换。同一份 `Path.of("src/main/java/com/agent/Agent.java")` 在 Windows 上被解析为三段名字，在 macOS 上同样如此；拼接用 `path.resolve("tool")` ，遍历用 `iterator()` / `getName(i)` ，比较用 `Path` 的语义，而不是字符串拼接 `"/"` 与 `"\\"` 。

`java.io.File` 的不足（都是它诞生于 Java 1.0 时代的历史包袱）：

1. **路径即字符串**： `File` 的很多操作本质是在字符串上做 `+` 与 `indexOf` ， `getParent()` 返回 `String` 而不是路径对象，导致「拼接后是否合法」没有类型保证，也很难写出平台无关的代码。
2. **错误处理不一致（最严重）**： `File.delete()` 、 `mkdir()` 、 `renameTo()` 等**失败时返回 `false` 而不抛异常**，调用方极易忘记检查；失败原因（不存在？无权限？目录非空？）完全丢失。NIO 的 `Files.delete()` 会抛出 `NoSuchFileException` / `DirectoryNotEmptyException` / `AccessDeniedException` 等具体异常——本实验的 `ReadFileTool` 正是靠这一点给出明确错误。
3. **缺少原子文件操作**： `Files.move(..., ATOMIC_MOVE)` 、 `REPLACE_EXISTING` 、 `COPY_ATTRIBUTES` 这类「要么完整成功、要么不动」的操作在 `File` 上没有对应能力，只能用「删除 + 重命名」凑，容易出现半个文件。
4. **缺少符号链接与属性支持**：读取/创建符号链接、读写 POSIX 权限与文件属性，需要 NIO 的 `Files.readAttributes` / `createSymbolicLink` 。
5. **没有虚拟文件系统（FileSystem）抽象**：NIO 可以把 ZIP 包当作一个文件系统来遍历（ `FileSystems.newFileSystem(zipPath)` ）， `Path` 在其中同样可用； `File` 只能表达真实磁盘路径。
6. **编码与资源管理不友好**：读写文本需要手工包 `InputStreamReader` 并处理编码；NIO 提供 `Files.readString` / `newBufferedReader` （默认 UTF-8）与 try-with-resources 友好的 `Stream<Path>` ——这正是本实验三个 Tool 的实现基础。

结论： `File` 并不是「在 Windows 上就不能用」，而是它的抽象层次太低（字符串 + 布尔返回值），把平台差异与错误处理的责任推给了每一个调用者。 `Path` / `Files` 把这些责任收回到 API 设计里：**平台差异由 JVM 处理，错误由异常明确表达，资源由 try-with-resources 保证释放**。这也解释了为什么本实验明确要求用 NIO 而不是 `File` 。

9.8 用 `@TempDir` 构造测试夹具而非依赖真实项目结构，带来了哪些好处？依赖真实目录会在 CI / 不同机器上出什么问题？

`@TempDir` 的好处（都在本实验的 32 个 Tool 用例中体现）：

1. **隔离性**：每个测试方法拿到一个全新的空目录，测试之间零共享状态，可任意顺序执行、可并行执行。
2. **确定性**：测试自己造数据（ `Files.writeString(dir.resolve("A.java"), "...")` ），期望值来自测试自身，而不是来自项目里恰好存在的内容。例如 `shouldFindKeywordWithLineNumberAndContent` 断言「第 2 行命中」，不需要关心仓库里其它文件有没有 `TODO` 。
3. **可写入且无副作用**：文件操作测试必须能造出「不存在」「不可解码」「深层嵌套」这些场景，只有临时目录可以随便写；同时它位于系统临时目录，不会污染工作区（本实验跑完测试后 `git status` 依然干净）。
4. **自动清理**：JUnit 在测试结束后递归删除临时目录，不需要手写 `finally` 清理，也不会在机器上积累垃圾。
5. **覆盖异常路径成为可能**：想测 `NoSuchFileException` ，只需指向临时目录里一个不存在的路径；想测「非法 UTF-8」，只需写几个非法字节 —— 如果依赖真实项目，这些场景根本无法安全构造。

如果测试依赖真实项目目录，会在 CI / 其它机器上出这些问题：

1. **工作目录不同**：Maven Surefire 的 `user.dir` 通常是模块目录，但在 IDE、Gradle、CI 容器里可能不同。测试里写 `Path.of("README.md")` 或 `Path.of("not-exist.txt")` 会随启动位置而异 —— 实验示例里那个 `readFile(Path.of("not-exist.txt"))` 就是这种脆弱写法（虽然它碰巧总是成立：该文件在项目根目录确实不存在）。
2. **环境缺少文件**：CI 上可能只 checkout 部分内容（稀疏检出、 `.gitignore` 排除、artifact 未生成），或换一台机器、换一个操作系统（路径分隔符、大小写敏感性不同），测试立刻失败——而且失败原因与代码质量无关。
3. **测试污染仓库**：为了让测试有数据可读，测试会往仓库里写文件；一旦中途失败，这些文件会留在工作区，导致 `git status` 变脏、下次提交误带垃圾文件，甚至影响其它测试。
4. **断言与真实内容耦合**：断言「 `README.md` 里有 5 行」这类事实，会在任何一次文档更新后失效；测试从「验证代码行为」退化成「验证项目内容」，维护成本高且信号价值低。
5. **互相干扰与顺序依赖**：多个测试写同一个真实目录时会互相踩踏，出现「单独跑过、一起跑失败」的偶发问题。

一句话：**测试应该验证代码的行为，而不是验证当前这台机器上的文件长什么样**。 `@TempDir` 提供了一块「可以随意破坏」的沙盒，让正常路径与异常路径都能被稳定、可重复地验证——这也是实验评分里「 `@TempDir` 夹具覆盖正常与异常路径、 `mvn test` 可重复」这一项的直接依据。

10. 选做与挑战任务完成情况

任务	内容	状态	证据
必做 1–3	`ListFilesTool` / `ReadFileTool` / `SearchCodeTool`	✅	`src/main/java/com/agent/tool/` 三个 Tool，签名与实验要求一致（`searchCode` 接收 `keyword` + `path`）
必做 4	每个 Tool 配 JUnit 测试（`@TempDir`）	✅	4 个 Tool 测试类、32 个 `@TempDir` 用例（工程总计 52 个）
必做 5	Git 增量提交	✅	`7e7179d lab02: file tools`
选做 1	`ListFilesTool` 最大深度参数	✅	`listFiles(root, maxDepth)` + `shouldRespectMaxDepth`
选做 2	`SearchCodeTool` 正则匹配	✅	`searchCodeRegex(Pattern, root)` + `shouldSupportRegexSearch`
选做 3	忽略列表抽为配置（`.gitignore` 风格）	✅	`.agentignore` + `IgnoreRules`（目录规则 / glob 规则）+ `IgnoreRulesTest` 5 个用例
挑战 1	`ListFilesTool` 树形结构输出	✅	`tree(root)` + `shouldRenderTreeWithIndentation` + Demo `--tree`
挑战 2	`ReadFileTool` 分页读取（`offset` + `limit`）	✅	`readFile(path, offset, limit)`（`BufferedReader` 流式）+ 4 个分页用例
挑战 3	预判 `String execute(String args)` 的序列化方案	✅ 已作答	§9.3：三种返回类型分别序列化为「每行一个路径」「原样文本」「`文件:行号: 内容`」，并用 `ToolResult` 统一成败

11. 实验总结

1. **「LLM ≠ 文件系统」是本实验最核心的认知。** 模型再强，它的世界也只有上下文窗口；项目代码要进入它的视野，必须由 Tool 把磁盘上的字节变成文本。这条边界决定了 Agent 架构的分工：**决策（概率）交给 LLM，执行（确定）交给 Tool**。
2. **文件操作的正确性必须靠测试来定义，而不是靠「跑一次试试」。** 本实验 32 个 Tool 用例里有一半是异常与边界路径：文件不存在、路径是目录、非法 UTF-8、不可解码文件、`target/` 内的 `.java`、空关键字、负深度、`limit <= 0`，以及编码在不同终端下的选择规则。这些都是「手跑一次 Demo 永远不会碰到、但线上一定会碰到」的场景。`@TempDir` 让它们可以在 0.2 秒内被反复验证。
3. **AI 生成的代码「结构对」但「边界糊」。** 三次 AI 任务的审查结论高度一致：AI 都很熟练地写出了 `Files.walk` + `try-with-resources` + `relativize` 的主干，但都在**边界**上偷懒 —— 忽略规则硬编码且语义混淆（`.DS_Store` 混进扩展名列表）、`Path::toString` 的平台差异、`catch (IOException ignored)` 静默吞掉一切、重载带来的调用歧义。这些都不是「语法错误」，编译器与 AI 自己都发现不了，只有把需求逐条拆成检查项（是否递归？是否过滤目录？失败是否中断？）才能审出来。
4. **异常设计是接口设计的一部分。** 这次特意区分了 `NoSuchFileException`（环境问题，可被调用方降级处理）与 `IllegalArgumentException`（调用方参数错误，必须立即暴露），并把「单文件失败跳过、整体失败报错」作为两条不同的策略固定下来。这为 Lab04 把异常统一包装成 `ToolResult(success=false)` 打好了基础。
5. **「先做对、再做统一」的顺序是对的。** 本实验刻意保留三个 Tool 签名不一致的技术债，用真实的痛点（写不出统一循环、没法统一生成 LLM 工具描述、异常契约各写一套、重载歧义）说明 Lab04 的 `Tool` 接口不是形式主义。抽象应当由痛点驱动，而不是先有接口再硬塞实现。
6. **平台无关性是可以「设计出来」的。** 用 `Path` 而不是字符串拼路径、在 Tool 输出边界统一 `/`、用 UTF-8 显式读写、用 `\n` 而非 `System.lineSeparator()` 做分页输出 —— 这些选择让同一份代码在 Windows 与 macOS 上行为一致，也顺手避免了「测试断言依赖平台」这一常见坑。

附录 A：Lab 02 新增/变更文件清单

新增
  .agentignore
  src/main/java/com/agent/tool/IgnoreRules.java
  src/main/java/com/agent/tool/ListFilesTool.java
  src/main/java/com/agent/tool/ReadFileTool.java
  src/main/java/com/agent/tool/SearchCodeTool.java
  src/main/java/com/agent/tool/FileToolsDemo.java
  src/test/java/com/agent/tool/IgnoreRulesTest.java
  src/test/java/com/agent/tool/ListFilesToolTest.java
  src/test/java/com/agent/tool/ReadFileToolTest.java
  src/test/java/com/agent/tool/SearchCodeToolTest.java
变更
  README.md（新增 File Tools 章节与工具签名表）

附录 B：常用命令

```
mvn clean test                                                  # 52 个测试
mvn -q -DskipTests compile
java -cp target/classes com.agent.tool.FileToolsDemo README.md TODO
java -cp target/classes com.agent.tool.FileToolsDemo --tree
java -cp target/classes com.agent.tool.FileToolsDemo README.md   # 中文输出已由 Demo 内部固定为 UTF-8
```

附录 C：V2 验收清单

V2
├── Java CLI ✅ Main / Agent（Lab01 回归测试 9 个用例通过）
├── ListFilesTool ✅ 递归 + 忽略规则 + 深度限制 + 树形输出
├── ReadFileTool ✅ UTF-8 + 明确异常 + 流式分页
├── SearchCodeTool ✅ 子目录 + 跳过二进制/忽略目录 + 文件:行号:内容 + 正则
├── Test ✅ 52 个用例全通过（Tests run: 52, Failures: 0, Errors: 0）
└── Git ✅ 7e7179d lab02: file tools